National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology

Lab Number:	07
Course Code:	CL2001
Course Title:	Data Structures - Lab
Semester:	Fall 2026
Instructor:	Muhammad Mehdi
Email:	muhammad.mehdi@nu.edu.pk

Guidelines

- You may use **only the concepts covered in the lab** to complete this task.
- Your submission must be a **single PDF file** that includes:
  - The source code (in plain text)
  - Screenshots of the corresponding outputs
- Name your file using the following format: **rollno_name_labtasknumber.pdf**
  Example: *25p-1234_ozair_labtask07.pdf*
- Unethical use of AI tools will result in **0 marks**.
- Late submissions will incur a penalty of **-3 marks per day**.

Tasks

ONLY USE POINTER NOTATION TO IMPLEMENT ALL THE SOLUTIONS. ARRAY INDEXING [ ] IS NOT ALLOWED.

1 Queue Using Stacks

Implement a **first-in-first-out (FIFO) queue using only two stacks**. The implemented queue should provide the basic operations of a normal queue: `enqueue`, `dequeue`, and `peek`.

Implement the following functions:

- `void enqueue(int x)`: Adds element `x` to the back of the queue.
- `int dequeue()`: Removes and returns the element at the front of the queue.
- `int peek()`: Returns the element at the front of the queue without removing it.

You must use only standard stack operations, which include pushing an element onto the top, accessing or removing the top element, and obtaining the stack size.

Example:

```
MyQueue myQueue;

myQueue.enqueue(1);     // queue is: [15]
myQueue.enqueue(2);     // queue is: [15 | 22]
```

```
myQueue.peek();         // returns 15
myQueue.dequeue();      // returns 15 (queue is now: [22])
```

2 Stack Using Queues

Implement a **last-in-first-out (LIFO) stack using only two queues**. The implemented stack should provide the basic operations of a normal stack: `push`, `pop`, and `peek`.

Implement the following functions:

- `void push(int x)`: Adds element `x` to the top of the stack.
- `int pop()`: Removes and returns the element at the top of the stack.
- `int peek()`: Returns the element at the top of the stack without removing it.

You must use only standard queue operations, which include inserting an element at the back, accessing or removing an element from the front, and obtaining the queue size.

Example:

```
MyStack myStack;

myStack.push(15);
myStack.push(22);

myStack.peek();     // returns 22
myStack.pop();      // returns 22 (stack is now: [15])
```

3 Hot Potato Game

Given a list of player names and an integer `k`, implement the **Hot Potato Game** using a queue. Place all players into a queue. Repeatedly remove the player at the front of the queue and move them to the back of the queue `k - 1` times. The player remaining at the front of the queue is then eliminated.

Continue this process until only one player remains. Return the name of the winning player. The program should display each eliminated player so that the complete elimination process can be observed.

The exact elimination order should be determined by the queue operations and the value of `k`.

Example:

```
Input:
players = ["Ali", "Ahmed", "Khalid", "Usman", "Farhan"]
k = 3
```

```
Output:
Eliminated: Khalid
Eliminated: Ali
Eliminated: Farhan
Eliminated: Ahmed

Winner: Usman
```

4 Students and Sandwiches

A school cafeteria offers two types of sandwiches: **circular** and **square**, represented by `0` and `1`, respectively. Students stand in a queue, and each student prefers one of the two sandwich types.

The number of sandwiches is equal to the number of students. The sandwiches are placed in a stack, with the first element representing the sandwich currently at the top.

At each step:

- If the student at the front of the queue prefers the sandwich at the top of the stack, the student takes the sandwich and leaves the queue.
- Otherwise, the student leaves the front of the queue and joins the end of the queue.
- The process continues until all students have received a sandwich or none of the remaining students wants the sandwich currently at the top of the stack.

Given the `students` queue and `sandwiches` stack, return the number of students who are unable to eat.

Example 1:

```
Input:
students =
front ->  [1 | 1 | 0 | 0]
sandwiches =
[0]  <- top
[1]
[0]
[1]

Output: 0
```

Explanation:

- The first student moves to the back of the queue because they prefer sandwich `1`, while the top sandwich is `0`.
- The second student also moves to the back for the same reason.

- The next student takes sandwich 0 and leaves the queue.
- The process continues until all students receive their preferred sandwich.

Therefore, the number of students unable to eat is 0.

Example 2:

```
Input:
students =
front ->  [1 | 1 | 1 | 0 | 0 | 1]
sandwiches =
[1]  <- top
[0]
[0]
[0]
[1]
[1]

Output: 3
```